

BuildStart Platform

WhatsApp Voice Message Analysis & Multilingual AI Agent Architecture

Target System: BuildStart Self-Hosted Infrastructure Date: August 31, 2026 Version: 1.0 (Production Architecture)

1. Executive Summary

The BuildStart WhatsApp Voice Message Analysis system enables conversational artificial intelligence to understand, transcribe, and intelligently respond to incoming customer voice recordings in real time. Rather than restricting customers to typed text, users can send audio voice notes (Push-to-Talk / `.oga` format) in **Sinhala (සිංහල)**, **Tamil (தமிழ்)**, or **English**.

The system downloads the voice recording over an isolated Docker network, performs acoustic transcription using high-accuracy multi-modal speech recognition (Google Gemini Flash), accesses the business knowledge base (product catalog, prices, policies, and FAQs), and returns a helpful, natural response back to the customer on WhatsApp within seconds.

2. Core Components of the Architecture

1. WhatsApp Ingress Gateway (WAHA)

Role: Manages the persistent WhatsApp Web WebSocket connection, device pairing via QR code, message reception, and outbound message delivery. It buffers incoming voice notes and provides authenticated HTTP endpoints for media downloads.

2. Edge Computing Runtime (Supabase Edge Functions)

Role: Serves as the orchestration layer running on Deno. It hosts serverless edge functions: `webhook-wsender` (ingress event handler), `process-message` (audio extraction & pipeline coordinator), and `ai-chat` (contextual business logic).

3. Multi-Modal Speech-to-Text Engine (Google Gemini Flash)

Role: Performs verbatim transcription of audio voice notes. Unlike traditional Speech-to-Text (STT) models that struggle with South Asian dialects, Gemini Flash natively understands acoustic audio streams in Sinhala, Tamil, English, and mixed colloquial speech (Singlish/Tanglish).

4. Business Knowledge Base & Database (PostgreSQL 17)

Role: Stores the store's dynamic data including products, storage and color variations, pricing, active FAQs, delivery rules, payment accounts, and full conversation histories.

5. AI Conversation & Reasoning Service (LLM Engine)

Role: Ingests the customer's transcribed text along with the real-time business catalog. It reasons over product availability, calculates delivery charges, formats prices, and generates conversational replies in the customer's native language.

3. Step-by-Step Voice Message Lifecycle

1 Customer Sends WhatsApp Voice Note

The customer records and sends a voice note to the business's linked WhatsApp number. WAHA receives the binary audio payload, generates a unique message ID, and emits an HTTP POST webhook event to the edge functions gateway.

2 Ingress & Voice Message Detection

The `webhook-wsender` function inspects the payload and detects voice signatures (e.g., `messageType: "ptt"` or `audioMessage` metadata). The message is enqueued into the database execution queue, and `process-message` is triggered.

3 Internal Container Audio Retrieval

The `downloadWahaAudio` module normalizes the media URL to the internal Docker network (`http://waha:3000`). Authenticating with the secret API key, it downloads the raw Opus/OGG audio stream directly into memory as a binary buffer (`Uint8Array`).

4 Speech-to-Text Transcription (Sinhala / Tamil / English)

The binary audio buffer is converted into Base64 format and sent to the **Google Gemini Flash** API. A specialized prompt instructs the engine to transcribe verbatim in the speaker's language and script without translating, summarizing, or adding meta-text.

5 Database Storage & Dashboard Synchronization

The transcribed text is saved into the PostgreSQL `conversations` table with metadata: `{ is_voice: true, transcription_provider: "gemini" }`. The web dashboard immediately renders the chat with a `Voice Note` pill and microphone icon.

6 AI Catalog Retrieval & Contextual Reasoning

The transcribed query is passed to `ai-chat`, which dynamically retrieves the business's products, variations, FAQs, and delivery settings. The AI matches the customer's intent (e.g., product pricing or store location) and generates a structured reply in the customer's language.

7 Automated WhatsApp Reply Dispatch

The generated answer (along with product images if applicable) is transmitted to WAHA via `POST /api/sendText`. WAHA delivers the reply message directly to the customer's WhatsApp conversation.

4. Multilingual Strategy: Sinhala, Tamil, & English

Supporting South Asian languages requires handling distinct phonetic structures, multiple scripts, and frequent code-mixing (such as Singlish or Tanglish).

Dimension	Traditional Whisper Model	Google Gemini Flash Multi-Modal
Sinhala (සිංහල)	High Word Error Rate (WER); struggles with colloquial grammar and complex consonants.	High Accuracy: Accurately transcribes native Sinhala script and colloquial phrasing.
Tamil (தமிழ்)	Moderate accuracy, often defaults to Indian Tamil phonetics.	High Accuracy: Robust comprehension of Sri Lankan Tamil accents and dialects.
Code-Mixing (Singlish/Tanglish)	Often fails or hallucinates non-existent words when English is mixed with native languages.	Native Understanding: Preserves English brand names (e.g., "iPhone 16", "256GB") embedded in native speech.
Processing Latency	3 to 7 seconds depending on model size and server GPU.	Under 1.5 seconds via Google's optimized tensor infrastructure.

5. Data Privacy & Audio Lifecycle Management

A critical design goal is maintaining low disk utilization while preserving necessary audit records:

Asset Type	Storage Location	Persistence State	Lifecycle & Cleanup Policy
Raw Audio Stream (Binary)	Edge Function RAM	Non-Persistent (Volatile)	Automatically garbage-collected immediately after transcription completes.
Temporary Audio File	WAHA Container Storage	Temporary Cache	Kept in container storage temporarily; automatically purged on container restart or cache rotation.
Transcribed Text	PostgreSQL <code>conversations</code>	Permanent	Retained for chat history, reporting, and future AI contextual memory.
Message Audit Record	PostgreSQL <code>message_queue</code>	Permanent	Preserves message ID, sender, timestamp, and duration for debugging.

6. Summary of Key Implementation Files

File Path	Primary Responsibility
supabase/functions/process-message/transcribe.ts	Downloads audio from WAHA, converts formats, and interfaces with Gemini Flash and Groq STT APIs.
supabase/functions/process-message/index.ts	Message queue worker that routes voice messages, saves records to PostgreSQL, and calls AI chat.
supabase/functions/ai-chat/index.ts	Injects product catalog, store FAQs, and delivery rules into the LLM system prompt.
frontend/src/pages/Conversations.tsx	Dashboard interface displaying live chat history with specialized Voice Note badges.
docker/docker-compose.override.yml	Declares the waha and functions services with internal Docker network bindings.
docker/.env	Secure configuration storing GEMINI_API_KEY , GROQ_API_KEY , and WAHA_API_KEY .

7. Conclusion

The BuildStart Voice Message Analysis architecture combines a self-hosted WhatsApp gateway (WAHA) with state-of-the-art multi-modal speech AI (Google Gemini Flash). By replacing complex monolithic infrastructure with modular serverless edge workers, the system achieves sub-2-second voice processing, native support for Sinhala, Tamil, and English, and automated AI sales engagement while keeping hosting overhead minimal.